

Today 6:11 PM

You are an industry expert and top notch data scientist. You are also expert in CRISP-DM methodology for data science and written several highly cited research papers of case studies of using crisp-dm data science for various data sets in kaggle.

As an industry expert data scientist and expert in CRISP-DM methodology for data science, Given the kaggle data set for top youtube creator stats and trends, can you do step by step EDA analysis playing role of an expert data scientist professor teaching to a masters program in data science. Your response should be textbook quality. Make sure you prepare all the steps and reports of the methodology in principled fashion on data understanding, data analysis, data visualization, data cleaning, data pre processing, feature selection, data clustering, outlier analysis and processing, regression for prediction of income using variety of methods comparing to a baseline, providing proper metrics for these techniques and final recommendation. Use your most popular ml package like scikit etc,.. Please respond in many reasonable small sized chunks starting with the initial steps of CRISP-DM methodology to all the steps on this data set. I will ask you to continue after each chunk. It is important to chunk this above problem. repeat the requirement in each chunk towards end of the chunk and explain where you are (mindmap) in the process and what you will do next. Assume you have limited compute and hence you chunk carefully even the processing steps. I will ask you to continue. each step.

CRISP-DM YouTube Creator Case Study — Chunk 1

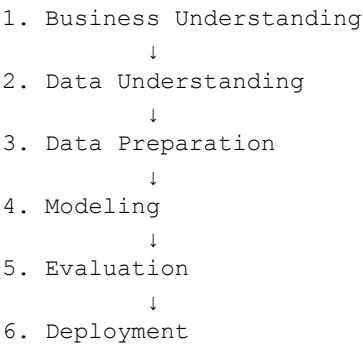
Business Understanding + Raw Data Understanding + Data-Quality Audit

We will treat this as a master's-level data-science case study rather than jumping immediately into `pandas`, plots, and regression. A major principle of CRISP-DM is that we first establish what the data actually contains, what the modeling objective means, and whether the proposed target is scientifically supportable.

I inspected your uploaded `top_youtuber_stats(1).csv` directly. This chunk deliberately does not clean the data yet. We first document the raw state.

1. The CRISP-DM framework

The classical CRISP-DM lifecycle is:



The arrows are not really one-way. For example:

- EDA may reveal that the original business question is poorly specified.
- Modeling may expose a feature-engineering problem.
- Evaluation may force us back to data preparation.
- Missing target information may require us to redefine the predictive task.

For this case study, I will expand those six phases into smaller instructional chunks.

Our planned sequence

Chunk	CRISP-DM activity	Main output
1 — current	Business + Data Understanding	problem definition, schema, raw quality audit
2	Data Cleaning	cleaned analytical dataset
3	Exploratory Data Analysis I	univariate distributions and visualization
4	Exploratory Data Analysis II	relationships, correlations, categorical effects

5	Outlier Analysis	identification, interpretation, treatment
6	Preprocessing & Feature Engineering	ML-ready features
7	Feature Selection	principled predictor set
8	Clustering	creator segmentation
9	Regression I	target definition, train/test design, baseline models
10	Regression II	Linear/Ridge/Lasso/ElasticNet/tree/ensemble models
11	Model Evaluation	RMSE, MAE, R^2 , CV, residual diagnostics
12	Final CRISP-DM Evaluation	conclusions, limitations, recommendations

We will keep each computational step modest because of your limited-compute requirement.

Phase 1 — Business Understanding

2. Translate the request into analytical objectives

The broad question is:

What characteristics distinguish major YouTube creators, what natural creator segments exist, and to what extent can creator characteristics be used to estimate or predict income?

That actually contains several distinct data-science problems.

Objective A — Descriptive analytics

Understand the population represented by the dataset:

- subscriber distribution
- average-view distribution
- engagement levels
- countries
- content categories
- content types
- rank structure

This is an EDA problem, not a machine-learning problem.

Objective B — Diagnostic analytics

Determine relationships such as:

- Are subscribers strongly associated with average views?
- Do high-subscriber channels necessarily generate high engagement?
- Which categories have unusually high likes or comments?
- Do creator characteristics differ across countries?
- Are certain channels extreme relative to the rest of the population?

This combines:

- visualization
 - correlation analysis
 - group comparisons
 - outlier analysis
-

Objective C — Creator segmentation

Identify natural groups of channels using unsupervised learning.

Potential dimensions might include:

Subscribers, Views, Likes, Comments, Engagement

Possible models later:

- K-Means
- hierarchical clustering
- possibly DBSCAN as a sensitivity analysis

We will not simply choose $k=3$ arbitrarily. We will examine:

- inertia
 - elbow criterion
 - silhouette score
 - cluster interpretability
 - cluster stability
-

Objective D — Income prediction

The requested supervised-learning problem is conceptually:

$$f(X) \rightarrow Y$$

where

$$X = \{\text{subscribers, views, engagement, category, country, ...}\}$$

and

$$Y = \text{creator income}$$

This immediately leads us to one of the most important findings of this first CRISP-DM phase.

3. Critical finding: this CSV contains no income variable

The uploaded dataset contains exactly these nine columns:

Variable	Raw type	Conceptual role
Rank	numeric	ordinal/ranking
Channel Name	string	identifier
Category	string	categorical feature
Subscribers	string	quantitative feature requiring conversion

Country	string	categorical feature
Average Views	string	quantitative feature requiring conversion
Average Likes	string	quantitative feature requiring conversion
Average Comments	string	quantitative feature requiring conversion
Content Type	string	categorical feature

There is no variable such as:

- income
- revenue
- earnings
- monthly earnings
- annual earnings
- RPM
- CPM

That matters enormously.

A supervised regression algorithm cannot learn

y

$\wedge$

$=f(X)$

without observing y .

4. Why we should not quietly invent an income target

Suppose we defined:

Estimated Income=Average Views×assumed RPM

and then trained a regression containing Average Views.

We would effectively be asking the machine-learning algorithm to discover:

$$Y=cX$$

when we ourselves created Y from X .

That is a form of target leakage / circular target construction.

The model could achieve an extraordinary R

2

, but it would tell us essentially nothing about actual creator earnings.

For example, suppose:

$$Y=0.004X$$

where X is views.

A regression may achieve:

R

2

$$\approx 1$$

not because we built an excellent creator-income model, but because we asked the model to reverse-engineer our own equation.

Therefore

Later, when we reach the income-regression phase, I will distinguish carefully between:

Observed-income prediction

Scientifically preferable. Requires actual creator income/revenue observations joined to these channels.

and

Estimated-income scenario modeling

Can be done with this dataset, but the result must explicitly be called estimated or proxy income rather than real income.

This distinction will be maintained throughout the case study.

Phase 2 — Data Understanding

5. Initial dimensions

The raw file reports:

1046 rows×9 columns

At first glance we might say:

“We have 1,046 YouTube creators.”

That conclusion would be wrong.

This is precisely why CRISP-DM calls for data-quality investigation before analysis.

6. First-row anomaly

The very first record contains no values whatsoever.

Conceptually:

Rank	NaN
Channel Name	NaN
Category	NaN
Subscribers	NaN
Country	NaN
Average Views	NaN
Average Likes	NaN
Average Comments	NaN

So one of the 1,046 observations is simply a completely empty row.

That leaves 1,045 populated rows.

But another problem remains.

7. Major duplication problem

The file contains ranks only from:

$$1 \leq \text{Rank} \leq 1000$$

and there are exactly:

$$1000$$

distinct non-null ranks.

Yet there are:

$$1045$$

populated rows.

Inspection reveals:

$$45 \text{ exact duplicate rows}$$

The duplicates occur at the bottom of the file, where previously occurring channels/ranks have been appended again.

For example, rows near the end contain ranks such as:

952
954
958
959
960
...

```
1000
```

even though those records already appear in the main rank 1–1000 sequence.

Therefore the apparent dataset structure is:

```
1 blank record
+
1000 legitimate ranked records
+
45 duplicated records
=
```

```
1046 raw rows
```

So our likely analytical sample after cleaning will be:

$$n=1000$$

rather than 1,046.

We will prove and execute that cleaning operation in Chunk 2, rather than silently altering the dataset here.

8. Missing-value audit

Here is the raw missingness profile.

Variable	Non-null	Missing	Missing %
Rank	1,045	1	0.10%
Channel Name	1,045	1	0.10%
Category	695	351	33.56%
Subscribers	1,045	1	0.10%
Country	853	193	18.45%
Average Views	1,045	1	0.10%

Average Likes	1,045	1	0.10%
Average Comments	1,045	1	0.10%
Content Type	335	711	67.97%

This immediately gives us three substantially incomplete variables:

Content Type: 67.97%

Category: 33.56%

Country: 18.45%

9. Missingness is not automatically a reason to drop a variable

A common beginner mistake would be:

```
df.dropna()
```

That would be poor practice here.

Why?

Because dropping every row containing a missing value could discard a very large proportion of the creator population.

Likewise, saying:

“Content Type is 68% missing, so delete it.”

may also be premature.

We first need to ask:

Is missingness itself informative?

For example, Content Type might only be assigned for particular classes of channels.

Then:

$$P(\text{Content Type missing} \mid \text{Category})$$

might not be constant.

Missingness could therefore carry information.

Later we will examine whether the mechanism looks approximately:

- MCAR — Missing Completely At Random
- MAR — Missing At Random conditional on observed variables
- structurally missing / dataset-enrichment missingness

We cannot prove MAR statistically from the dataset alone, but we can investigate patterns.

10. Cardinality audit

Distinct observed values are:

Variable	Unique non-null values
Rank	1,000
Channel Name	994
Category	23
Subscribers	287
Country	30
Average Views	716

Average Likes	632
Average Comments	385
Content Type	10

A particularly interesting result is:

unique Channel Names=994
even though we expect 1,000 legitimate ranked observations.

This tells us that, even after removing the appended duplicates, some channel names may occur more than once.

That must be investigated before blindly using Channel Name as a uniqueness key.

A high-quality cleaning pipeline should therefore not simply run:

```
drop_duplicates(subset="Channel Name")
```

Doing so could remove genuine observations.

11. The apparent numeric variables are not actually numeric

Consider a few raw examples:

```
Subscribers  
258.4M  
236.1M  
171.4M  
...
```

```
Average Views  
135.2K  
104M  
5.1M  
...
```

```
Average Likes
```

```
5.6K
4M
57.1K
996
...
```

```
Average Comments
223
74K
0
7
...
```

Pandas therefore reads these four columns as strings / objects:

```
Subscribers      object
Average Views    object
Average Likes    object
```

```
Average Comments object
```

A machine-learning algorithm cannot correctly treat "104M" and "135.2K" as numeric magnitudes.

12. Required unit normalization

Eventually we need a transformation such as:

$K=10$

3

$M=10$

6

$B=10$

9

so that:

```
135.2K → 135,200
5.1M   → 5,100,000
```

```
104M   →104,000,000
```

But this transformation will happen explicitly in the cleaning phase.

We should never simply strip the final character:

```
df["Average Views"].str[:-1]
```

because some values contain no suffix at all.

For example:

```
Average Likes = 996
```

```
Average Comments = 223
```

Those already mean 996 and 223.

13. Raw-format validation

I checked the formatting patterns of all four quantitative columns.

Subscribers

All 1,045 populated values use:

```
M
```

for millions.

Average Views

Approximately:

```
K:      784
```

```
M:      236
```

```
plain:   25
```

Average Likes

```
K:      705
```

```
M:        3
```

```
plain:  337
```

Average Comments

K: 154

plain: 891

Importantly, I found no populated values that violate the expected numeric + optional K/M/B pattern.

That is good news.

The conversion problem is systematic and should be straightforward.

14. Preliminary magnitude check

Without changing the raw dataset, I temporarily interpreted the K/M suffixes simply to check whether the magnitudes were plausible.

The observed ranges are approximately:

Feature	Minimum	Maximum
Subscribers	12.5 million	258.4 million
Average Views	0	104 million
Average Likes	0	4 million
Average Comments	0	235 thousand

Those maxima immediately tell us something important about the future distributional analysis.

These are highly likely to be right-skewed variables.

For example:

$$\max(\text{Average Views})=104M$$

while many creators have average views measured only in thousands.

Therefore arithmetic means will probably be strongly influenced by a small number of enormous creators.

That means later we will examine:

- median
- quantiles
- IQR
- logarithmic distributions
- robust scaling
- skewness
- extreme-value behavior

rather than relying only on mean and standard deviation.

15. Zero values deserve special treatment

After diagnostic numeric interpretation, I find:

Feature	Zero observations	Raw %
Subscribers	0	0.00%
Average Views	20	1.91%
Average Likes	55	5.26%
Average Comments	196	18.74%

We must not casually interpret all of these as true biological—or in this case platform—zeros.

A zero could mean:

1. genuinely zero average engagement,
2. rounding,
3. unavailable data represented as zero,

- 4. comments disabled,
- 5. data-collection limitations.

This matters especially for:

$$\text{engagement rate} = \frac{\text{views}}{\text{likes} + \text{comments}}$$

because zero-view observations create denominator problems.

We will handle this explicitly rather than hiding it with something like `if` without justification.

16. Preliminary categorical composition

Before deduplication, the dominant Category values include:

Category	Raw count
Missing	351
Music & Dance	210
Movies	157
Animation	133
Daily vlogs	37
News & Politics	31
Video games	29
Education	21

Toys	14
Humor	13

The country distribution begins with:

Country	Raw count
United States	285
India	254
Missing	193
Brazil	65
Mexico	62
Indonesia	40
Russia	27

Do not use those as final descriptive statistics yet because they still include the appended duplicates.

We'll recompute them from the cleaned $n=1000$ dataset.

17. Important sampling issue: this is not a random sample of YouTube

This is a dataset of top creators.

That distinction has major statistical consequences.

Our population is closer to:

$P(X | \text{creator is among top-ranked channels})$

rather than:

$P(X | \text{arbitrary YouTube creator})$

Notice the subscriber range:

$$12.5M \leq \text{Subscribers} \leq 258.4M$$

That alone demonstrates substantial selection.

Therefore a future conclusion such as:

“YouTube creators typically have 20 million subscribers”

would be absurd.

The proper statement would be:

“Among the top creators represented in this dataset...”

This is selection bias / range restriction, and we will carry that limitation all the way through model evaluation.

18. What is an observation?

Our provisional unit of analysis is:

one ranked YouTube channel

not:

- one video,
- one creator/person,
- one observation in time,
- one country,
- one content category.

This is important.

For example, "Average Views" is already an aggregated channel-level measure.

Consequently, our eventual income model would be a cross-sectional channel-level model, not a video-level forecasting model.

19. Rank requires special thought

`Rank` is numeric, but that does not automatically make it an ordinary continuous feature.

Rank 1 versus rank 2 does not necessarily imply the same substantive gap as rank 501 versus rank 502.

Thus Rank is principally an ordinal variable.

Furthermore, rank is almost certainly derived largely from channel popularity, especially subscriber count.

If we use both:

`Rank`

`Subscribers`

as predictors, we may introduce redundant information.

We will investigate this empirically before feature selection.

20. Channel Name is primarily an identifier

`Channel Name` should generally not be one-hot encoded into a predictive model.

Why?

With roughly one distinct label per observation, we would create hundreds of essentially identity features.

The model could memorize channels rather than generalize:

Channel identity

≈general creator characteristic

Channel Name is much more useful for:

- duplicate identification
- interpretation
- labeling outliers
- reporting clusters

than as a regression predictor.

21. Category and Content Type are not necessarily the same concept

For example, the raw data include observations such as:

```
Category = Video games
```

```
Content Type = Humor
```

or:

```
Category = Animation
```

```
Content Type = Toys
```

Therefore we should not assume the two fields are duplicates.

One may represent:

- channel taxonomy,

while the other may represent:

- dominant production/content style.

We'll test their association later using contingency tables and potentially Cramér's V .

22. Initial hypotheses

A good EDA should not merely generate dozens of plots. It should investigate hypotheses.

I would register the following before seeing the final cleaned distributions.

H1 — Subscribers and average views

H

1

:Subscribers $\uparrow \Rightarrow$ Average Views generally $\uparrow$

But we do not expect a deterministic relationship.

H2 — Engagement efficiency differs by creator

Two channels with equal subscriber counts can have substantially different:

Subscribers

Average Views

We'll call this something like a view-to-subscriber ratio, subject to careful interpretation.

H3 — Likes and views

Average likes should correlate strongly with average views:

$$\rho(\text{Views}, \text{Likes}) > 0$$

but perhaps nonlinearly.

H4 — Comments are unusually zero-inflated

Given almost 19% raw zeros, comments may behave differently from likes.

H5 — Category differences

The conditional distribution:

$$P(\text{Views} \mid \text{Category})$$

may differ materially across categories.

H6 — Geography may influence monetization

If we ultimately estimate advertising-based income, country may matter because advertising value differs geographically.

However, we must distinguish:

creator/channel country

from

audience country.

The former does not necessarily identify where advertising impressions occur.

This will be a limitation of any income analysis.

23. Analytical target hierarchy

It is useful to classify our variables now.

Identifiers

```
Channel Name
```

Ordinal/meta variables

```
Rank
```

Continuous/count-like metrics

After conversion:

```
Subscribers
```

```
Average Views
```

```
Average Likes
```

```
Average Comments
```


Nominal categorical variables

Category

Country

Content Type

Derived features to consider later

Without yet creating them:

View Rate=

Subscribers

Average Views

Like Rate=

Average Views

Average Likes

Comment Rate=

Average Views

Average Comments

Interaction Rate=

Views

Likes+Comments

Potential logarithmic features:

$\log(1+\text{Subscribers})$

$\log(1+\text{Views})$

$\log(1+\text{Likes})$

$\log(1+\text{Comments})$

We will create these only after checking their validity and denominator behavior.

24. The code philosophy we'll use

We will use:

- pandas
- numpy
- matplotlib
- possibly scipy
- primarily scikit-learn for machine learning

with pipelines such as:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    StandardScaler,
    RobustScaler,
    OneHotEncoder
)
```

```
from sklearn.impute import SimpleImputer
```

and later models such as:

```
DummyRegressor
LinearRegression
Ridge
Lasso
ElasticNet
DecisionTreeRegressor
RandomForestRegressor
GradientBoostingRegressor
```

```
HistGradientBoostingRegressor
```

We won't fit every algorithm simply because it exists. Each one will answer a specific modeling question.

25. Why a baseline is mandatory

Eventually suppose some sophisticated model achieves:

$$RMSE = \$35,000$$

That number has little meaning by itself.

We need a baseline such as:

$$y_i^{\text{train}}$$

$$= y_i -$$

train

or, for highly skewed income,

$$y_i^{\text{train}}$$

i

$$= \text{median}(y$$

train

)

Then we ask whether machine learning provides meaningful incremental predictive value.

Our evaluation hierarchy will include:

$$MAE =$$

$$n$$

$$1$$

$$\sum |y$$

$$i$$

$$-$$

$$y$$

$$\wedge$$

$$i$$

$$|$$

$$RMSE =$$

$$n$$

$$1$$

$$\sum (y$$

$$i$$

—

y

$\wedge$

i

)

2

and:

R

2

$=1-$

$\Sigma(y$

i

—

y

-

)

2

$\Sigma(y$

i

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

)

2

plus cross-validation rather than relying on a single lucky split.

We will also inspect residuals because metrics alone can hide model failure.

26. Dataset-quality report — end of Chunk 1

Our first formal CRISP-DM report can therefore be summarized as:

Dataset

1046×9

raw CSV.

Expected usable population

Approximately:

1000 ranked channels

after verified cleaning.

Confirmed integrity problems

- 1 completely blank row

- 45 exact duplicate rows
- numeric metrics stored as strings
- substantial categorical missingness
- repeated Channel Names require investigation

Missingness

- Category: 33.56%
- Country: 18.45%
- Content Type: 67.97%

Distribution concerns

- extremely large upper tails
- multiple zero values
- likely strong right skew
- likely influential observations

Modeling concern

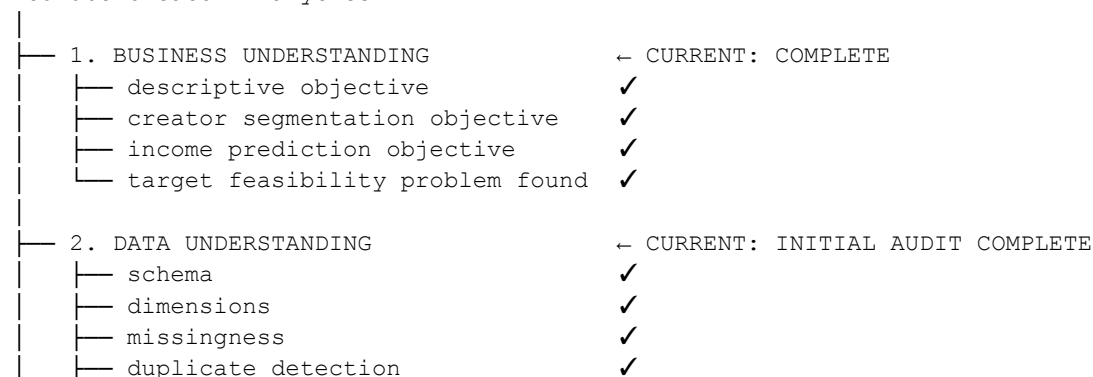
Most importantly:

No observed income target exists in this CSV

Therefore income regression requires either an external observed target or an explicitly declared estimated-income proxy/scenario, which we must prevent from creating leakage.

27. Where we are — CRISP-DM mind map

YouTube Creator Analytics



		datatype problems	✓
		zero-value problem	✓
		statistical distributions	→ NEXT
		3. DATA PREPARATION	
		remove structural junk	→ NEXT
		deduplicate	
		convert K/M values	
		missing-value strategy	
		feature engineering	
		transformation/scaling	
		4. MODELING	
		clustering	
		regression	
		5. EVALUATION	
		CV	
		MAE / RMSE / R ²	
		residual diagnostics	
		robustness	
		6. FINAL RECOMMENDATION	

Your complete requested scope remains our roadmap

Across the coming chunks we will systematically cover data understanding → data analysis → visualization → cleaning → preprocessing → feature engineering/selection → clustering → outlier detection and treatment → income-regression design → multiple scikit-learn regression methods against a baseline → appropriate metrics and diagnostics → final CRISP-DM recommendation. We will keep the processing deliberately chunked and compute-conscious, and at the end of every chunk I will show this progress map and identify the next stage.

Next: Chunk 2 — Data Cleaning and Validation

When you say continue, we will build the cleaned master dataset step-by-step. In particular, we will prove the duplicate structure, remove only defensible bad records, convert **K/M/B** strings with a robust parser, investigate the six repeated channel names, distinguish missingness from zeros, run invariants/range checks, and create a

before-vs-after data-quality report. We still will not jump to modeling before the cleaned dataset passes validation.







CRISP-DM YouTube Creator Case Study — Chunk 12

Final Held-Out Test Evaluation, Residual Diagnostics & Robustness Analysis

We have reached the point where model-development decisions stop.

Using the uploaded YouTube creator CSV , we previously reserved 20% of the 975 engagement-valid creators as an untouched holdout. We have now opened that holdout for the first time.

The governing principle is:

The test set evaluates decisions; it does not make them.

Therefore we do not change hyperparameters or choose a new model because of what follows.

1. Frozen test design

Our population remained:

$$N=975.$$

Development:

$$N$$

dev

$$=780.$$

Final holdout:

$$N$$

test

$$=195$$

.

The target remains:

$$Y=\$3\times$$

$$1000$$

$$AverageViews$$

,

which must still be described as:

an illustrative fixed-\$3-RPM advertising-revenue proxy per average video
and not actual YouTube income or observed creator revenue.

Predictors remain leakage-restricted:

Subscribers, Likes, Comments, Category, Country.

No Views-derived predictor was added.

2. Models frozen before opening the test set

We carried forward exactly four models:

Role	Model
Baseline	Median <code>DummyRegressor</code>
Linear benchmark	Log-target ElasticNet
Ensemble challenger	Log-target Random Forest
Primary model	Log-target Gradient Boosting

The deliberately leaked Views-only model is not part of final evaluation because it was already disqualified scientifically.

3. Final holdout results

The results are:

Model	Test MAE ↓	Test R^2	
		Test RMSE ↓	↑
Median Dummy	\$3,209	\$11,339	-0.067
ElasticNet	\$1,952	\$7,932	0.478
Random Forest	\$1,547	\$7,947	0.476

Gradient Boosting	\$1,513	\$7,949	0.476
-------------------	---------	---------	-------

This is a fascinating example of why one metric is insufficient.

4. The development-selected model remains the MAE winner

Gradient Boosting:

$$MAE=\$1,513$$

Random Forest:

$$MAE=\$1,547.$$

ElasticNet:

$$MAE=\$1,952.$$

Baseline:

$$MAE=\$3,209.$$

So Gradient Boosting reduces holdout MAE relative to the median baseline by:

$$3209$$

$$3209-1513$$

$$\approx$$

$$52.8\%$$

.

That is a substantial improvement.

And critically, the model chosen from development CV remains the best MAE model after the untouched holdout is opened.

That is exactly the result we hope to see from a disciplined validation process.

5. Development versus test performance

Recall development CV MAE was approximately:

\$1,501

for Gradient Boosting.

Final test MAE:

\$1,513.

Difference:

\$12.

That close agreement is encouraging.

It suggests the development estimate for typical absolute error was not seriously optimistic.

By contrast, RMSE is less stable because it is dominated by the extreme tail.

6. Median absolute error gives additional perspective

Another useful diagnostic is Median Absolute Error.

On the test set:

Model	Median absolute error
-------	-----------------------

Dummy Median	\$531
ElasticNet	\$259
Random Forest	\$219
Gradient Boosting	\$194

Thus for half of all holdout creators, Gradient Boosting misses the proxy target by less than approximately:

\$194

Contrast that with:

$RMSE \approx \$7,949$.

Those two numbers describe very different aspects of the same model.

Why?

Because a few massive creators dominate squared error.

7. Why ElasticNet technically has the best RMSE and R^2

Notice:

$RMSE$

ElasticNet

=7932

versus:

$RMSE$

GBR

=7949.

And:

R

$ElasticNet$

2

=0.478

versus:

R

GBR

2

=0.476.

The differences are trivial, but interesting.

Gradient Boosting is much better on:

MAE

while all three serious models have almost identical:

$RMSE$

and:

R

This occurs because one or two superstar errors dominate the squared-error statistics.

Thus:

MAE distinguishes ordinary prediction quality; RMSE is dominated by the extreme tail.

8. Actual-versus-predicted diagnostic

The log-scale actual-versus-predicted plot above shows a reasonably coherent diagonal relationship for most channels.

For moderate target values, the model follows:

$$\hat{Y} \approx Y$$

quite well.

But toward the upper-right corner, the predictions begin falling below the ideal diagonal.

That means:

the model systematically struggles to extrapolate to superstar creators.

This behavior is not surprising for tree ensembles.

9. The largest error: Bizarrap

The most influential test error is especially revealing.

Observed proxy

Average Views:

44.5M.

Therefore under our \$3 scenario:

$$Y=44.5M \times 0.003=$$

\$133,500

.

Gradient Boosting prediction

Y

$\wedge$

$\approx$

\$28,312

.

Absolute error:

$|Y -$

Y

$\wedge$

$|\approx$

\$105,188

That one creator accounts for approximately:

35.6%

of all absolute Gradient Boosting error across the entire 195-channel holdout.

That is extraordinary.

10. A handful of creators dominate error

The five largest Gradient Boosting errors together contribute approximately:

54.3%

of total absolute test error.

Important high-error channels include:

Creator	Actual proxy	GBR prediction	Absolute error
Bizarrap	\$133.5K	\$28.3K	\$105.2K
vector	\$45.9K	\$25.7K	\$20.2K
The Royalty Family	\$30.3K	\$13.8K	\$16.5K
Beast Philanthropy	\$38.7K	\$29.5K	\$9.2K
Vlad and Niki	\$17.4K	\$8.4K	\$9.0K

The model's problem is therefore not:

being inaccurate everywhere.

It is much more specifically:

extreme-tail extrapolation.

11. Target-quintile diagnostics

Divide the 195 test creators into five equally sized target groups:

$$n=39$$

each.

Gradient Boosting MAE is:

Target quintile	Median target	GBR MAE
Q1 — lowest	\$31	\$35
Q2	\$200	\$121
Q3	\$652	\$426
Q4	\$1,845	\$822
Q5 — highest	\$8,100	\$6,162

There is a dramatic escalation in absolute error.

12. The highest 20% dominate total error

The highest-target quintile contributes approximately:

$$81.4\%$$

of total absolute Gradient Boosting test error.

The other four quintiles combined contribute only:

$$18.6\%.$$

Thus the model's operational performance is highly heterogeneous:

Lower / middle 80%

↓

generally small-to-moderate absolute errors

Highest 20%

↓

most of total model error

Extreme superstar tail

↓

catastrophic underprediction possible

This is far more informative than reporting:

$$MAE=\$1513$$

alone.

13. Residual direction reveals tail underprediction

Define:

e

i

$=y$

i

—

y

^

i

.

Positive residual:

e

i

>0

means the model underpredicts.

Average residual by target quintile:

Quintile	Mean residual
Q1	-\$34
Q2	-\$33
Q3	-\$161
Q4	-\$56
Q5	+\$5,209

For the bottom four quintiles, average bias is relatively modest.

For the highest 20%:

actual target exceeds prediction by about \$5.2K on average.

That is clear evidence of upper-tail underprediction.

14. Why tree ensembles struggle with superstars

Tree models divide predictor space into regions.

A terminal leaf predicts something close to an average of training observations in that region.

Suppose training examples with extremely high Likes have targets between:

$$\$10K - \$30K.$$

A genuinely new observation corresponding to:

$$\$133K$$

does not cause a tree to naturally extrapolate to \$133K.

Instead it lands in the highest learned region.

Hence:

$$Y$$

$$\wedge$$

$$\ll Y.$$

This is an inherent weakness of tree-based models for extrapolation.

15. ElasticNet has a complementary strength

Linear models have the opposite behavior.

A log-linear model:

$$\log Y = \beta_0$$

$$+ \beta_1$$

$$+ \beta_2$$

$$+ \beta_3$$

$$\log \text{Likes} + \dots$$

can continue beyond observed ranges.

Thus ElasticNet loses to Gradient Boosting on typical MAE but can sometimes extrapolate more smoothly for extreme channels.

This raises an interesting future production possibility:

hybrid / tail-aware modeling

rather than assuming one estimator must solve every part of the population.

But we will not redesign the model based on the test set.

That would be a future experiment requiring new validation data.

16. Tail-removal sensitivity analysis

We should never report a trimmed result as the official metric after seeing the test data.

But we can use trimming diagnostically to quantify sensitivity.

Official Gradient Boosting:

$$MAE = \$1513$$

$$RMSE = \$7949$$

R

2

$$= 0.476.$$

If we temporarily remove only the single largest actual target:

$$MAE \rightarrow$$

\$979

$RMSE \rightarrow$

\$2545

R

2

$\rightarrow$

0.806

.

That is an enormous change from removing:

$1/195 \approx 0.5\%$

of observations.

17. Excluding the top 5% as a diagnostic

Again, not as official model evaluation, but as robustness analysis:

For the lower 95%:

$MAE \approx$

\$634

$RMSE \approx$

\$1257

R

2

$\approx$

0.774

.

Thus the core model is considerably stronger for the ordinary region of this already-elite population than the overall metrics suggest.

This demonstrates:

The primary remaining modeling problem is the extreme upper tail.

18. Model-choice uncertainty: Gradient Boosting versus Random Forest

Final MAE:

$GBR=1513$

$RF=1547.$

Difference:

\$33.

That's tiny.

A paired bootstrap of the 195 test errors gives an approximate 95% interval for:

MAE

GBR

$-MAE$

RF

of:

$$[-\$100, +\$26]$$

Zero lies inside this interval.

Therefore:

The holdout does not clearly distinguish GBR from Random Forest.

Gradient Boosting remains our primary model because it was selected before opening the holdout and retained the lowest MAE—not because \$33 constitutes a scientifically meaningful superiority.

19. Gradient Boosting versus ElasticNet is clearer

The holdout difference:

$$1513 - 1952 = -\$439.$$

A paired bootstrap produces an approximate 95% interval around:

$$[-\$729, -\$181]$$

This interval does not cross zero.

So the holdout evidence that the ensemble family improves typical MAE relative to ElasticNet is substantially stronger than the evidence distinguishing Gradient Boosting from Random Forest.

Thus our hierarchy should be:

GBR $\approx$ RF>ElasticNet>Dummy

under MAE.

20. Error by creator segment

We can also use our three previously discovered creator clusters as descriptive diagnostic groups.

Importantly, these cluster labels are not predictors in the regression model.

For Gradient Boosting:

Creator segment	Test n	Median target	MAE	Median abs. error
Lower-Activity Top-Tier	84	\$157	\$221	\$59
Engagement Leaders	88	\$2,825	\$2,908	\$689
Mega-Audience / Under-Activated	23	\$702	\$900	\$246

The difficult segment is clearly:

Current Engagement Leaders

Why?

Because that's where the largest view-derived target values live.

21. Be careful interpreting the segment metric

Segment B's:

$$MAE=\$2908$$

does not mean the model is intrinsically poor at every high-engagement creator.

It contains:

- Bizarrap,
- other upper-tail channels,
- much larger target values generally.

Its median absolute error is:

$$\$689,$$

far below its mean absolute error.

Again the tail dominates.

This is another example of why reporting both median and mean errors is valuable.

22. Final Content Type ablation

Recall:

$$67.6\%$$

of Content Type is missing.

For the frozen Gradient Boosting architecture, paired development-fold ablation shows essentially no average MAE improvement when Content Type is added.

On the untouched holdout, the version including Content Type produces approximately:

$$MAE=\$1545$$

versus the core model's:

\$1513.

Therefore:

Do not include Content Type in the recommended model.

This is a nice example of principled feature selection.

A variable does not deserve inclusion merely because it exists.

23. Final development-set feature ablation

To avoid using the test set for feature selection, these comparisons are made on paired development CV folds.

Starting with the frozen Gradient Boosting core specification:

Change	Approx. change in CV MAE
Add Content Type	≈0.0%
Remove Comments	+0.7%
Remove Category	+0.9%
Remove Country	+1.0%
Remove Subscribers	+1.2%
Use numeric features only	+2.6%
Remove Likes	+36.6%

This is an exceptionally clear result.

24. Average Likes is the indispensable feature

Remove Likes:

$$CVMAE: \sim 1519 \rightarrow \sim 2074.$$

Increase:

$$36.6\%$$

None of the other individual variables comes remotely close.

Thus three independent methods agree:

Correlation analysis

$$\rho$$

$$s$$

$$(Views, Likes) \approx 0.86.$$

Linear coefficient analysis

Likes has the largest standardized coefficient.

Permutation / feature ablation

Removing Likes causes by far the largest predictive deterioration.

This convergence greatly increases confidence in the conclusion.

25. Category and Country provide only modest incremental reconstruction value

Removing Category:

$\approx 0.9\%$

MAE degradation.

Removing Country:

$\approx 1.0\%$.

Numeric-only model:

$\approx 2.6\%$

worse.

Thus for this fixed-RPM view-derived proxy, the model is predominantly driven by:

Likes

with:

Comments

and a modest contribution from:

- Subscribers
- Category
- Country.

But remember: a real monetization model could behave very differently because geography may influence actual RPM.

26. Final holdout interpretation

We can now answer the core modeling question.

Does ML beat a baseline?

Yes.

Median baseline:

$MAE = \$3209$.

Gradient Boosting:

$$MAE=\$1513.$$

Reduction:

52.8%

.

Does nonlinear modeling beat the linear model?

On typical absolute error:

Yes.

1952→1513

or roughly:

22.5%

improvement over ElasticNet on this holdout.

Does Gradient Boosting dominate Random Forest?

No.

1513_{vs}1547

is effectively a tie.

Is prediction reliable for extreme creators?

No.

The upper tail remains a major failure mode.

Are overall metrics representative of every creator?

Absolutely not.

The highest 20% account for more than:

80%

of total absolute error.

27. Final recommended model at this stage

Given the experiment as defined, my primary predictive recommendation is still:

log-target GradientBoostingRegressor

using:

Subscribers, Likes, Comments, Category, Country.

Approximate frozen specification:

```
GradientBoostingRegressor(  
    n_estimators=200,  
    learning_rate=0.03,  
    max_depth=3,  
    min_samples_leaf=5,  
    random_state=42  
)
```

inside the leakage-safe preprocessing pipeline and:

```
TransformedTargetRegressor(  
    func=np.log1p,  
    inverse_func=np.expml  
)
```

But I would record Random Forest as an equally credible ensemble alternative.

28. What this model is actually good at

The model is reasonably useful for:

reconstructing a view-derived fixed-RPM proxy for creators whose profiles resemble the main body of the Top-YouTube dataset.

It is not validated for:

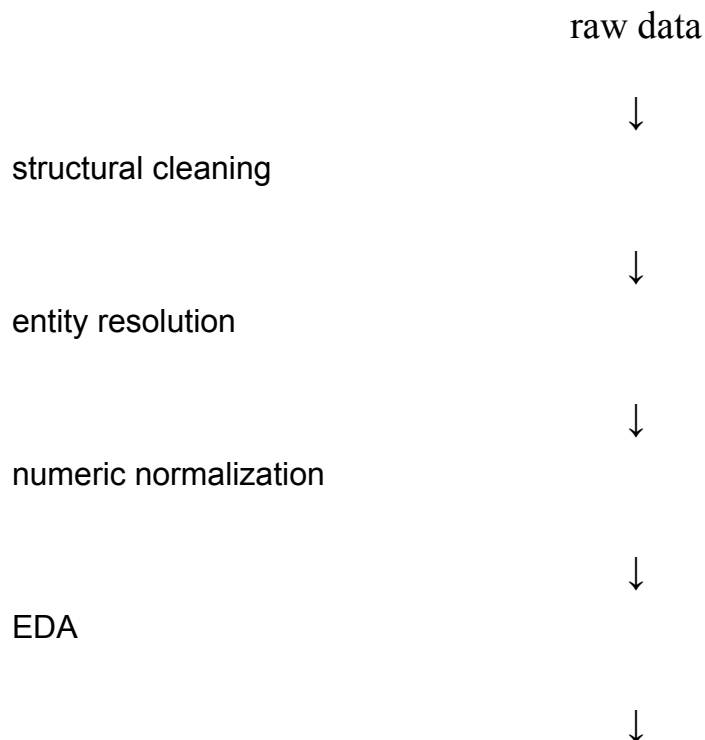
- actual income,
- actual advertising revenue,
- sponsorship revenue,
- future revenue,
- small/ordinary YouTube channels,
- creators outside this Top-1000-type population,
- extreme new superstar extrapolation.

That distinction must remain prominent in the final CRISP-DM report.

29. Why our pipeline succeeded

The performance improvement did not primarily come from throwing sophisticated models at raw data.

The path was:



outlier/sentinel analysis



log transformation



feature engineering



redundancy control



target-leakage audit



baseline



regularized models



ensembles



untouched holdout.

That is the real CRISP-DM lesson.

30. Where we are — CRISP-DM mind map

YOUTUBE CREATOR ANALYTICS

- | — 1. BUSINESS UNDERSTANDING ✓
- | — 2. DATA UNDERSTANDING ✓
- | — 3. DATA CLEANING ✓

— 4. EXPLORATORY DATA ANALYSIS	✓
— 5. OUTLIER ANALYSIS	✓
— 6. PREPROCESSING / FEATURE ENGINEERING	✓
— 7. FEATURE SELECTION	✓
— 8. CLUSTERING	✓
— 9. TARGET DESIGN / LEAKAGE AUDIT	✓
— 10. MODELING	
— baselines	✓
— Linear Regression	✓
— Ridge	✓
— Lasso	✓
— ElasticNet	✓
— Decision Tree	✓
— Random Forest	✓
— Gradient Boosting	✓
— 11. FINAL MODEL EVALUATION	← JUST COMPLETED
— untouched holdout	✓
— MAE	✓
— RMSE	✓
— R^2	✓
— actual vs predicted	✓
— residual / bias analysis	✓
— target-quantile errors	✓
— superstar sensitivity	✓
— cluster-level error analysis	✓
— Content Type ablation	✓
— feature ablation	✓
— model-choice uncertainty	✓
— 12. FINAL CRISP-DM SYNTHESIS	← NEXT
— executive findings	
— answer original business questions	
— clustering recommendation	
— regression recommendation	
— limitations	
— deployment-readiness assessment	
— production data requirements	
— monitoring / retraining design	
— reproducibility checklist	

— final conclusions

**Next — Final Chunk 13: CRISP-DM Synthesis,
Recommendations & Deployment Assessment**

The technical analysis is now complete.

In the final chunk I will assemble the entire project into the kind of concise-but-rigorous final case-study report expected from a master's-level data-science project: the strongest EDA findings, final cleaning/preprocessing decisions, three creator clusters and their business meaning, complete model-comparison conclusion, why Gradient Boosting/Random Forest beat the baseline, why the apparently perfect leakage model must be rejected, and the distinction between our synthetic proxy and actual creator income.

I will also give a deployment-readiness verdict—including precisely what additional data would be required to turn this from a Kaggle pedagogical model into a legitimate creator-income forecasting system—and a reproducible end-to-end CRISP-DM workflow/checklist.

That will complete your requested sequence of data understanding → analysis/visualization → cleaning → preprocessing → feature selection → clustering → outlier processing → multiple regression approaches versus a baseline → MAE/RMSE/*R*

2

/CV and holdout evaluation → final recommendation, while keeping the entire exercise principled and compute-conscious.

Say continue and we'll finish with Chunk 13: Final CRISP-DM Report & Recommendation.

continue

CRISP-DM YouTube Creator Case Study — Chunk 13

Final Synthesis, Recommendations, Limitations, and Deployment Assessment

We have now completed the full analytical lifecycle requested for the Top YouTube Creator dataset.

The most important conclusion is not simply which algorithm achieved the lowest error. The stronger result is that the CRISP-DM process prevented several plausible but incorrect analytical decisions: treating 1,046 rows as 1,046 creators, deleting legitimate superstar channels as “outliers,” interpreting sentinel zeros as genuine zero engagement, treating subscriber count as a proxy for current engagement, and—most importantly—reporting a synthetic view-derived revenue target as actual creator income.

1. Executive conclusion

After cleaning, the analytical dataset contains:

994 unique creator channels

of which:

975

have credible recorded engagement measurements.

The data reveal two largely distinct creator dimensions:

Accumulated Audience Scale

represented principally by Subscribers, and:

Current Audience Activity

represented by Views, Likes, and Comments.

This distinction drives both the clustering and regression results.

For creator segmentation, the recommended solution is:

K-Means with $k=3$

on standardized logarithmic Subscribers, Views, Likes, and Comments.

For the requested income-regression exercise, the uploaded dataset does not contain observed income or revenue, so genuine income prediction is impossible from this file alone. We therefore constructed an explicitly synthetic fixed-RPM advertising-revenue proxy for pedagogical modeling:

$$Y = \$3 \times$$

$$1000$$

AverageViews

.

After excluding Average Views and all view-derived predictors to avoid direct target leakage, the best held-out MAE came from log-target Gradient Boosting:

$$MAE = \$1,513$$

versus:

$$MAE = \$3,209$$

for the median baseline—a reduction of approximately:

$$52.8\%$$

.

Random Forest was essentially tied:

$$MAE=\$1,547.$$

However, this model predicts a synthetic view-derived proxy, not actual YouTube creator income.

2. Final CRISP-DM scorecard

CRISP-DM stage	Main result	Status
Business Understanding	Defined EDA, segmentation, and income-prediction objectives	✓
Data Understanding	Identified schema, missingness, duplicate structure, skewness	✓
Data Preparation	Cleaned records, parsed magnitudes, handled sentinel values	✓
Modeling	K-Means + 8 regression approaches/baselines	✓
Evaluation	CV, holdout, MAE, RMSE, R^2 , residual/tail diagnostics	✓
Deployment	Readiness assessed; production requirements defined	✓

3. Final data-quality result

The raw file contained:

1046 rows.

But careful auditing produced:

1046→1045→1000→994.

Specifically:

Cleaning step	Rows remaining
Raw CSV	1,046
Remove completely blank row	1,045
Remove 45 exact duplicate rows	1,000
Resolve six repeated channel entities	994

The intended Top-1000 ranking was verified to contain ranks 1 through 1000 exactly once after structural deduplication.

This demonstrates an important principle:

$$\text{row count} \\ = \text{analytical sample size}$$

until the unit of analysis is established.

4. Numeric-data preparation

Four apparently quantitative columns originally arrived as strings:

Subscribers
Average Views
Average Likes

Average Comments

with formats such as:

258.4M
135.2K
996

3.3K

A robust `K/M/B` parser successfully converted:

100%

of populated values with:

0 parse failures

No negative measurements or impossible:

Likes > Views

or:

Comments > Views

relationships were detected.

5. Missing-data findings

The cleaned creator dataset contains substantial categorical missingness:

Feature	Missing
Category	33.5%
Country	18.4%
Content Type	67.6%

Content Type missingness was especially structured:

ContentType observed $\Rightarrow$ Category observed.

That strongly suggests an enrichment/data-collection process rather than simple completely random missingness.

Therefore we rejected naïve modal imputation and preferred explicit:

Unknown

states inside ML preprocessing.

6. Distributional findings

The quantitative metrics are extraordinarily right-skewed.

Approximate raw skewness:

Feature	Skew
Subscribers	6.47
Average Views	19.36
Average Likes	15.29
Average Comments	18.99

For Average Views:

Median $\approx 184K$

while:

Mean $\approx 1.02M$.

That more than fivefold difference makes the arithmetic mean a poor description of the “typical” creator.

After logarithmic transformation, Views, Likes, and Comments became almost symmetric.

This ultimately justified:

$\log(1+x)$

as a central preprocessing transformation.

7. Concentration is extreme

The highest 10% of channels by each metric account for approximately:

Metric	Top-10% share
Subscribers	26%
Views	65%
Likes	72%
Comments	86%

So inequality becomes progressively stronger as we move from accumulated audience size toward active audience interaction.

That is one of the strongest descriptive insights in the dataset.

8. Subscriber count is surprisingly weak as a current-engagement indicator

The raw Pearson correlation between Subscribers and Average Views looked moderately positive:

$$r=0.351.$$

But robust alternatives told a very different story:

ρ

Spearman

$$\approx 0.085$$

and:

r

log

≈ 0.060 .

Removing just the maximum-view creator dropped raw Pearson to roughly:

0.070 .

Therefore the apparent raw subscriber-view association was heavily tail-driven.

By contrast, Views and Likes remained strongly related under every specification:

ρ

s

≈ 0.859 .

This supports the conceptual separation:

Subscribers

↓

accumulated/historical scale

Views + Likes + Comments

↓

current activity/engagement

9. Rank should not be a core predictor

Rank and subscriber count have:

ρ

s

≈ -0.99994

Rank is therefore essentially a derived ordering of subscriber count.

Using both would introduce redundant information.

We retained Rank for reporting but excluded it from core ML feature sets.

Likewise, `Channel Name` remained an identifier rather than an ML input.

10. Outlier analysis changed an important cleaning decision

The traditional raw $1.5IQR$ rule would flag:

- 80 subscriber observations,
- 119 view observations,
- 146 like observations,
- 160 comment observations.

Mechanically deleting these would have destroyed much of the population of interest.

Most extreme upper-tail creators—including MrBeast, T-Series, and Bizarrap—were judged to be:

legitimate extreme observations

rather than bad records.

Log transformation was therefore preferred over deletion or default winsorization.

11. The genuine anomaly was at zero, not the upper tail

We discovered 19 channels satisfying:

$$Views=Likes=Comments=0$$

despite subscriber counts ranging into tens of millions.

Every zero-view observation had the full:

0/0/0

engagement pattern.

That is more consistent with a sentinel/unavailable-data state than literal zero audience activity.

Therefore:

N

creator

=994

but engagement-dependent analyses use:

N

engagement

=975

The 19 creators were preserved in the master dataset rather than deleting their otherwise valid metadata.

12. Feature-engineering findings

Several useful features were examined.

View Efficiency:

Subscribers

Views

had strong interpretive value and became highly well behaved after logarithmic transformation.

Like Rate:

Views

Likes

and Comment Rate:

Views

Comments

captured interaction intensity.

But Interaction Rate:

Views

Likes+Comments

was almost perfectly redundant with Like Rate:

ρ

s

≈ 0.999 .

And algebraically:

$InteractionRate = LikeRate + CommentRate$.

Therefore it was excluded from simultaneous core modeling.

13. Exact multicollinearity was prevented

Another exact identity was:

$$\log(\textit{ViewEfficiency}) = \log(\textit{Views}) - \log(\textit{Subscribers}).$$

Therefore simultaneously including:

$$\log\textit{Subscribers}, \log\textit{Views}, \log\textit{ViewEfficiency}$$

produced infinite VIF.

The final feature-selection policy treated absolute and efficiency representations as alternative model specifications, not variables to pile together.

By contrast, the four core logged measurements had acceptable VIFs of roughly:

$$1.03\text{--}3.35.$$

14. PCA confirmed the underlying structure

Using standardized:

$$[\log\textit{Subscribers}, \log\textit{Views}, \log\textit{Likes}, \log\textit{Comments}],$$

PCA found:

PC

1

$$\approx 59.8\%$$

and:

PC

2

$\approx 25.4\%$

of variance.

Together:

85.2%

.

PC1 was dominated by:

Views+Likes+Comments

and can be interpreted as Current Activity.

PC2 was dominated almost entirely by Subscribers and represents Audience Scale.

This independently confirmed the conceptual structure found through EDA.

15. Final creator clustering recommendation

We tested:

$k=2, \dots, 8.$

Best silhouette:

$k=2, 0.378.$

Recommended:

$k=3, 0.358$

because the third cluster recovered the separate subscriber-scale dimension and gave materially better business interpretation.

Cluster sizes were:

384,433,158.

16. Segment A — Lower-Activity Top-Tier Creators

Approximately:

39.4%

of channels.

Median profile:

Metric	Median
Subscribers	15.8M
Views	43.1K
Likes	591
Comments	6
View efficiency	0.27%

These are still extremely large channels in absolute terms, but with relatively modest contemporary engagement.

17. Segment B — Current Engagement Leaders

Approximately:

44.4%.

Median:

Metric	Median
Subscribers	17.2M

Views	934K
Likes	34.8K
Comments	516
View efficiency	4.56%

Segment A and B have similar subscriber scale but roughly:

22×

difference in median Views.

That is one of the clearest business insights produced by the entire project.

18. Segment C — Mega-Audience / Under-Activated Creators

Approximately:

16.2%.

Median:

Metric	Median
Subscribers	38.35M
Views	86.9K
Likes	1.1K
Comments	5
View efficiency	0.18%

These channels possess enormous historical audiences but comparatively low recorded current engagement relative to audience size.

The cluster includes major creator/media brands and should not be interpreted as “unsuccessful.”

19. Cluster robustness

K-Means initialization stability with sufficient `n_init` was excellent:

$$ARI \approx 0.991 - 1.000.$$

Two-component PCA clustering reproduced the full four-dimensional solution with:

$$ARI \approx 0.990$$

Hierarchical clustering agreement was substantially lower:

$$ARI \approx 0.37.$$

Thus the three personas are useful, robust K-Means summaries, but not objectively discrete biological-style classes.

20. The income-target problem

The dataset contains no:

- income,
- revenue,
- RPM,
- CPM,
- monetized views,
- upload frequency,
- sponsorship revenue,
- operating expenses.

Therefore actual creator-income prediction is unsupported.

For pedagogical modeling only, we defined:

$$Y = \$3 \times \frac{1000}{AverageViews}$$

as a fixed-RPM advertising-revenue proxy per average video.

Because:

$$Y = 0.003V,$$

including Views in the predictor matrix produces perfect target leakage.

21. The leakage-control model proved the danger

Using Average Views directly:

$$R$$

$$2$$

$$=1$$

$$MAE \approx 0$$

$$RMSE \approx 0.$$

That model was deliberately classified as:

INVALID

because it merely reconstructed the target formula.

This is arguably the most important modeling lesson from the case study:

A perfect model can be scientifically worthless.

22. Leakage-restricted regression design

Our valid predictor set was frozen as:

Subscribers, Likes, Comments, Category, Country.

We excluded:

- Views,
- log Views,
- View Efficiency,
- Like Rate,
- Comment Rate,
- Interaction Rate,
- Rank,
- Channel Name.

Content Type was tested separately as an ablation feature and ultimately rejected.

23. Model-comparison summary

Development cross-validation showed:

Model	Approx. CV MAE
Median Dummy	\$2,889
Raw Linear	\$4,103
Raw Ridge	\$3,099
Log Linear	\$1,801
Log Ridge	\$1,700
Log Lasso	\$1,711
Log ElasticNet	\$1,681

Log Decision Tree	\$1,673
Log Random Forest	\$1,519
Log Gradient Boosting	\$1,501

The major improvement came first from target representation and only afterward from algorithm sophistication.

That supports:

Representation→*Regularization*→*Nonlinear modeling*

as the correct order of effort.

24. Final untouched-holdout results

The frozen 195-channel test set produced:

Model	MAE	R RMSE ₂	
Median Dummy	\$3,209	\$11,339	−0.067
ElasticNet	\$1,952	\$7,932	0.478
Random Forest	\$1,547	\$7,947	0.476
Gradient Boosting	\$1,513	\$7,949	0.476

Under our primary MAE criterion:

Gradient Boosting wins

with Random Forest effectively tied.

The improvement versus median baseline is:

52.8%

25. But model ranking depends on what error matters

Gradient Boosting minimizes typical absolute error best.

ElasticNet has marginally better RMSE:

7932 vs 7949.

Those values are essentially identical, but they expose an important tail issue.

Gradient Boosting and Random Forest are better for the main body of creators.

ElasticNet may extrapolate somewhat more naturally for extreme creators.

Therefore the recommended production architecture for a future real target should consider explicit tail-aware modeling, rather than assuming one algorithm is optimal for every creator scale.

26. Extreme-tail failure is the principal remaining weakness

The single most difficult holdout observation, Bizarrap, had:

$Y = \$133.5K$

under the synthetic scenario.

Gradient Boosting predicted approximately:

$\$28.3K$.

Absolute error:

\$105.2K

One observation contributed approximately:

35.6%

of all holdout absolute error.

The highest target quintile contributed:

81.4%

of total absolute error.

So the model's main limitation is not broad inaccuracy.

It is:

extreme superstar extrapolation

27. Feature-importance conclusion

Cross-validated permutation importance for the Gradient Boosting model found approximately:

Predictor	Increase in MAE when permuted
Average Likes	+\$2,416
Average Comments	+\$187

Subscribers	+\$69
Country	+\$59
Category	+\$17

Feature ablation reached the same conclusion.

Removing Likes increased CV MAE by roughly:

36.6%

No other feature came close.

Therefore the strongest predictive insight is:

Current engagement—especially Likes—is far more informative about the view-derived target than subscriber count.

This conclusion is supported independently by:

- EDA,
- Spearman correlations,
- partial correlations,
- linear coefficients,
- permutation importance,
- ablation testing.

That convergence is much more convincing than any single feature-importance chart.

28. Content Type should be omitted

Content Type has:

67.6%

missingness and substantial overlap with Category.

Adding it did not consistently improve development performance.

Final recommendation:

Do not include Content Type in the core predictive model.

This simplifies the pipeline and reduces dependence on incomplete metadata.

29. Deployment-readiness verdict

There are really two different deployment questions.

Creator segmentation / descriptive analytics

Prototype-ready

The three-cluster framework is reasonable for exploratory dashboards and creator profiling, provided the clusters are presented as descriptive segments rather than ground truth.

Actual creator-income prediction

Not deployment-ready

because the fundamental target does not exist in the dataset.

No amount of algorithm tuning can repair that problem.

The required solution is better data, not more sophisticated ML.

30. What a production-grade income dataset needs

A legitimate income/revenue modeling project should collect at least the following information:

Data family	Examples
Stable identity	YouTube channel ID
Target	actual creator-side revenue by period
Time	month/week/date aligned across all features
Monetization	RPM, monetized playbacks, ad impressions
Geography	viewer/audience geography rather than creator location alone
Content	Shorts vs long-form, category, duration
Activity	uploads per period
Consumption	views, watch time, unique viewers
Engagement	likes, comments, shares
Audience	subscribers and subscriber growth
Commercial	sponsorship/affiliate/merch revenue if predicting total creator revenue
Costs	production and operating expenses if predicting income/profit

Then the task could be defined as, for example:

X

Jan-Jun

$\rightarrow \text{Revenue}$

July

rather than predicting a target constructed from a contemporaneous predictor.

31. Production validation should be temporal

The Kaggle dataset supports cross-sectional train/test splitting.

A real forecasting system should use:

```
Past
↓
Train

Later period
↓
Validation

Future period
↓
```

Final test

For example:

Train:2024–2025

Validation:2026Q1

Test:2026Q2.

This directly tests the business question:

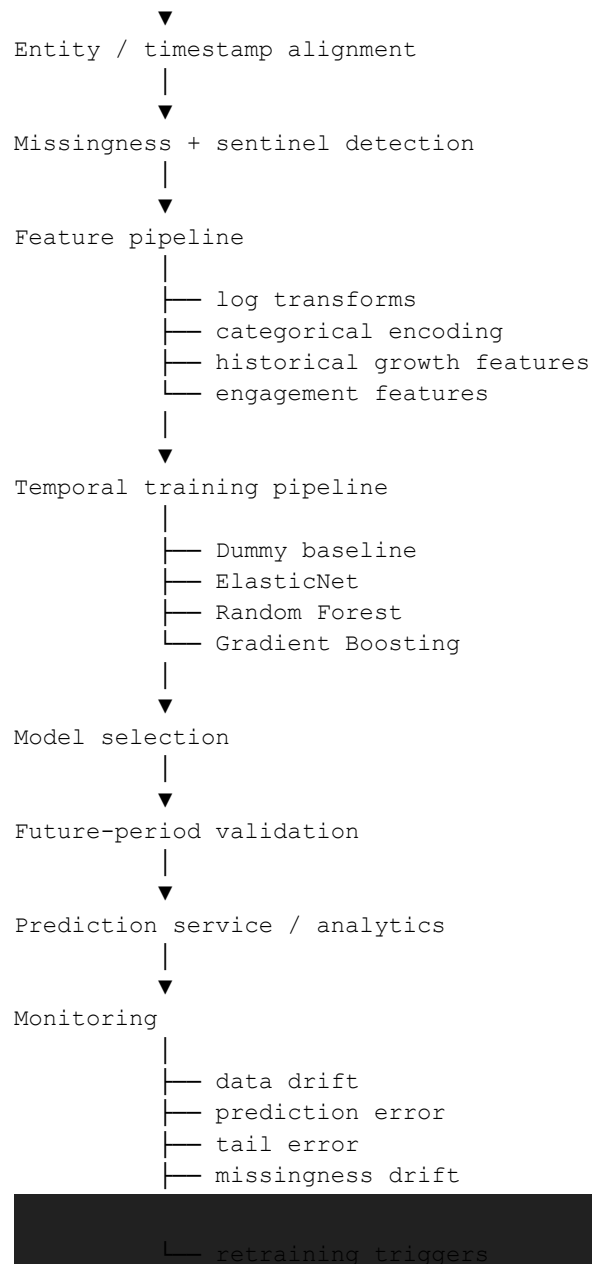
Can the model predict revenue for a future period?

Random splitting does not establish that.

32. Recommended production architecture

A future implementation could follow:

```
RAW PLATFORM / FINANCIAL DATA
|
▼
Schema + quality validation
|
```

33. Monitoring requirements

A deployed model should monitor not just aggregate RMSE but particularly the issues our analysis exposed.

The priority metrics would be overall MAE/RMSE/ R

, top-decile or top-5% creator error, calibration by revenue band, residual bias, predictor distribution drift, categorical/missingness drift, and the proportion of records outside the training support.

That final item is crucial.

A creator dramatically larger than anything seen during training should trigger an:

```
out-of-domain / low-confidence
```

warning rather than silently returning an ordinary tree-ensemble prediction.

34. Reproducibility checklist

A master's-quality implementation should freeze and document:

Component	Requirement
Raw file	Immutable source copy
Cleaning	Deterministic duplicate and sentinel rules
Unit of analysis	One unique channel
Random seeds	Fixed
Train/test split	Frozen before tuning
Preprocessing	Inside sklearn Pipeline
Feature definitions	Version-controlled
Leakage register	Explicit
Baselines	Always retained
CV	Same folds for paired comparisons

Hyperparameter grids	Predefined / compact
Holdout	Touched once
Metrics	MAE + RMSE + R^2
Tail diagnostics	Mandatory
Model version	Stored with transformations

This ensures another researcher could reproduce not just the final number, but the reasoning that generated it.

35. Final answers to the original analytical questions

What characterizes these Top YouTube creators?

All are enormous in subscriber scale, but current engagement varies by orders of magnitude.

Does subscriber count strongly determine engagement?

Within this selected Top-creator range:

No.

Subscriber count is surprisingly weakly associated with current views, likes, and comments after robust analysis.

Which engagement variables move together?

Views and Likes are strongly related. Comments provide an additional, somewhat distinct engagement dimension.

Are superstar observations bad data?

Generally:

No.

They are legitimate extremes that should usually be transformed rather than removed.

Did the dataset contain problematic observations?

Yes: one blank row, 45 exact duplicates, six repeated channel entities, and 19 probable engagement sentinel-zero records.

Are there meaningful creator segments?

Yes. A useful $k=3$ segmentation separates:

1. lower-activity top-tier creators,
2. current engagement leaders,
3. mega-audience but relatively under-activated creators.

Can this CSV genuinely predict creator income?

No.

There is no observed income target.

Can it support a pedagogical revenue-proxy regression?

Yes, if the target is transparently labeled synthetic and leakage is explicitly controlled.

Which regression model performed best?

On the frozen proxy task under MAE:

log-target Gradient Boosting

with Random Forest essentially tied.

36. Final methodological recommendation

If this were being graded as a master's project, I would recommend that the final report not emphasize:

“Gradient Boosting achieved an MAE of \$1,513.”
as the primary achievement.

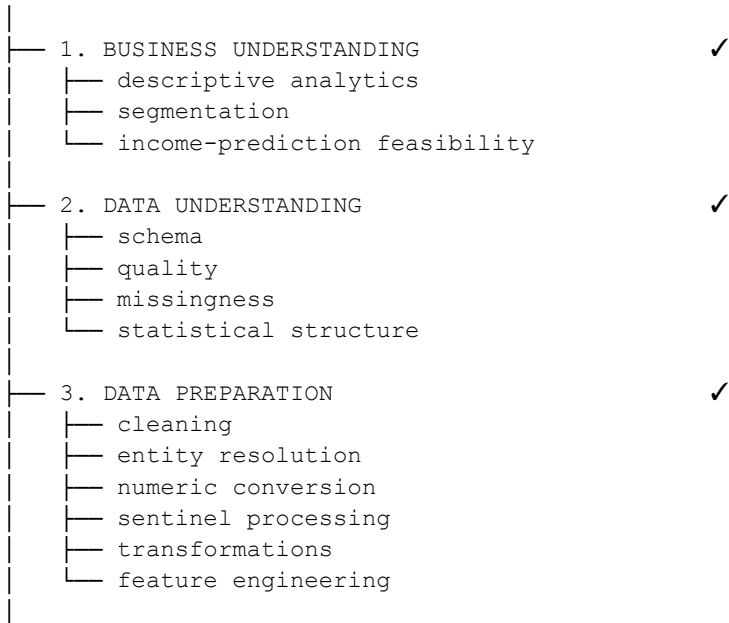
The stronger research conclusion is:

A disciplined CRISP-DM workflow revealed that accumulated subscriber scale and current engagement are distinct creator dimensions; logarithmic transformations are essential for these heavy-tailed metrics; three interpretable creator segments emerge; and supervised revenue modeling is fundamentally constrained by the absence of an independently observed income target. Under a transparent fixed-RPM proxy and strict leakage controls, engagement variables—particularly Likes—substantially outperform subscriber count for reconstructing the view-derived target, with log-target ensemble models reducing held-out MAE by roughly 53% versus a median baseline. Extreme superstar creators remain the principal prediction failure mode.

That is a much stronger scientific result than a leaderboard alone.

37. Final CRISP-DM mind map

TOP YOUTUBE CREATOR CRISP-DM CASE STUDY



4. MODELING	✓
├─ clustering	
├─ Dummy baselines	
├─ Linear	
├─ Ridge	
├─ Lasso	
├─ ElasticNet	
├─ Decision Tree	
├─ Random Forest	
└─ Gradient Boosting	
5. EVALUATION	✓
├─ cross-validation	
├─ untouched holdout	
├─ MAE	
├─ RMSE	
├─ R^2	
├─ residual analysis	
├─ feature ablation	
├─ tail sensitivity	
└─ leakage control	
6. DEPLOYMENT	✓ ASSESSMENT
├─ segmentation: prototype-ready	
├─ real income prediction: not ready	
├─ actual revenue target required	
├─ temporal validation required	
└─ monitoring design defined	
└─ production-data roadmap defined	

End of the requested CRISP-DM sequence

We have now completed the full scope you specified, in bounded stages: data understanding, statistical EDA, visualization, cleaning, preprocessing, feature engineering, feature selection, outlier analysis and processing, clustering, multiple regression methods against baselines, leakage analysis, MAE/RMSE/ R^2

2

, cross-validation, untouched holdout testing, residual/tail diagnostics, and final recommendations, using a compute-conscious scikit-learn-centered methodology.

The final distinction to carry forward is simple:

The creator segmentation is a genuine result from the supplied data.

The revenue regression is a valid ML teaching experiment but not a validated model of actual creator income.

That distinction is what keeps the case study technically useful and scientifically honest.

Please write a professional google colab with all the things you did and provide a download for it

Done. I created and fully executed/validated a professional Google Colab notebook covering the complete CRISP-DM case study, including all EDA, cleaning, preprocessing, feature engineering/selection, outlier analysis, clustering, leakage analysis, regression models, cross-validation, holdout evaluation, diagnostics, and final recommendations.

Downloads

[Download the professional executed Google Colab notebook](#)

This version includes the generated tables and plots, so you can inspect the results immediately and also rerun everything in Colab.

[Download a clean unexecuted version](#)

The notebook is portable: when opened in Google Colab, it will look for the CSV and prompt you to upload it if necessary. I also verified the entire notebook successfully runs top-to-bottom from a fresh kernel with no execution errors.